NapCart Architecture Spec v1

System Architecture Specification

Version: v1.0 Date: 2026-05-28 Status: Draft Author: Codex with Naptime AI

1. Purpose

This document defines the target MVP architecture for NapCart after approval of:

- NapCart Project Roadmap
- NapCart PRD v1
- NapCart ERD v1

It translates the approved product scope into a practical technical design that can be implemented step by step.

2. Architecture Goals

The architecture must:

- support one real restaurant launch first
- remain multi-tenant-ready for future restaurant onboarding
- keep restaurant staff WhatsApp-first
- keep the dashboard and database as the system of record
- support standalone storefront deployment now
- remain API-first for future integration into existing restaurant websites
- avoid overengineering while preserving clean upgrade paths

3. Launch Context Defaults

The first production target for NapCart is:

- country: Pakistan
- currency: PKR
- default language: English
- default timezone: `Asia/Karachi`

- distance unit: kilometers
- default phone normalization target: Pakistan mobile format with `+92`

Architecture note:

- these are launch defaults, not hardcoded product limitations
- the schema and config model must still support future region expansion

4. Architecture Principles

- Modular monolith first, not microservices
- Server-owned data access, not direct client-to-database business logic
- Tenant and branch awareness in every operational workflow
- Integration adapters for external providers
- Snapshot important business data at order time
- Keep operational truth in the backend even when staff work from WhatsApp
- Use configuration over custom code when onboarding new restaurants later

5. Recommended Stack

5.1 Core Stack

- Frontend framework: `Next.js`
- Language: `TypeScript`
- Runtime/deployment: `Vercel`
- Database: `PostgreSQL`
- Database platform: `Supabase Postgres`
- ORM: `Prisma`
- Admin auth: `Supabase Auth`
- Media storage: `Supabase Storage`

5.2 Why This Stack Fits NapCart

- `Next.js` supports storefront, admin dashboard, route handlers, and API-first evolution in one codebase
- `TypeScript` reduces integration and domain-model mistakes
- `PostgreSQL` is ideal for relational commerce and operational data
- `Prisma` gives fast MVP development with clear schema control

- `Supabase` gives production-ready Postgres, auth, and storage without forcing us into a no-code workflow
- `Vercel` fits the application deployment model well for a modern Next.js app

6. High-Level Architecture Choice

6.1 Architecture Style

NapCart should be built as a `modular monolith`.

This means:

- one main application codebase
- one primary database
- clear internal modules
- external integrations behind service boundaries

Why this is correct for MVP:

- faster to build and operate
- easier to reason about
- lower DevOps complexity
- enough flexibility for multi-tenant growth

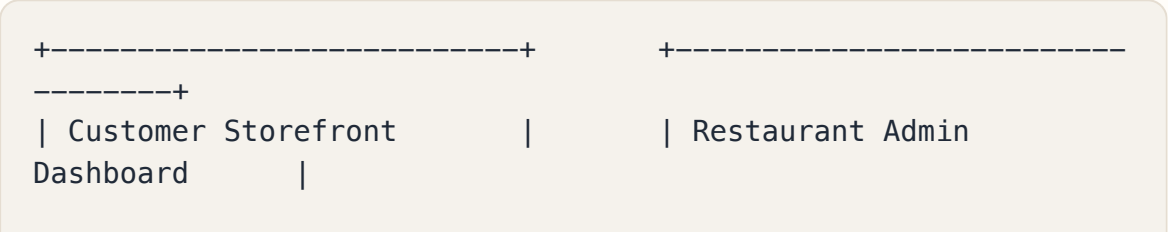
6.2 Future Evolution Path

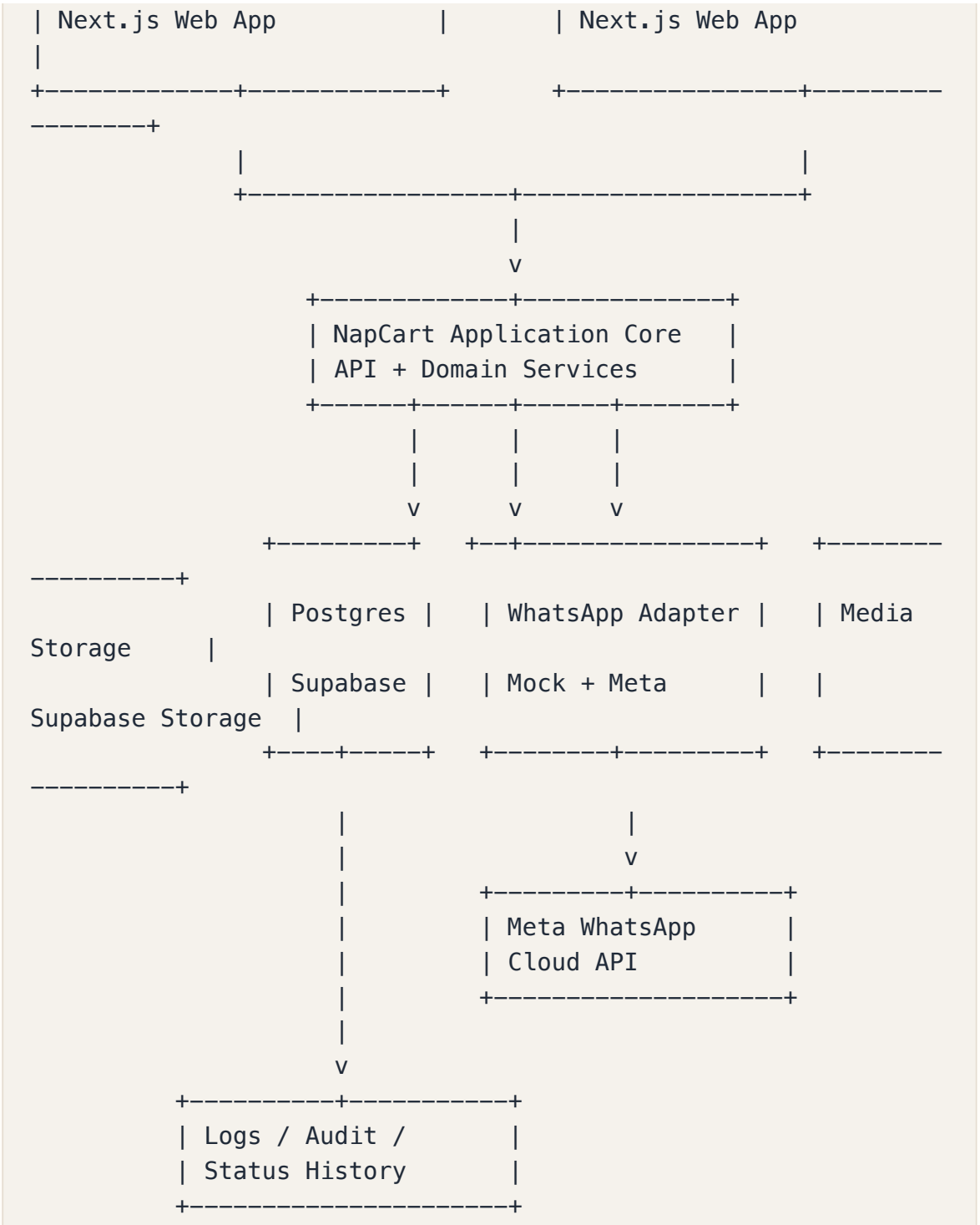
If scale later requires it, NapCart can split selected capabilities into separate services:

- WhatsApp messaging processor
- analytics/reporting worker
- media processing
- external integration gateway

That is not needed for MVP.

7. System Context





8. Application Layers

NapCart should be structured into the following layers.

8.1 Presentation Layer

Contains:

- storefront UI
- checkout UI

- admin dashboard UI
- admin settings screens

Responsibilities:

- collect validated input
- render tenant and branch-specific data
- call server APIs and server actions
- never own critical business rules by itself

8.2 API / Transport Layer

Contains:

- public storefront endpoints
- admin endpoints
- webhook endpoints
- internal route handlers

Responsibilities:

- request parsing
- authentication / authorization entry
- validation
- response shaping
- handoff to domain services

8.3 Domain Service Layer

Contains the real application logic:

- branch selection logic
- menu retrieval logic
- delivery eligibility logic
- order creation logic
- customer matching logic
- WhatsApp send / receive orchestration
- audit/status updates

This is the most important layer in the system.

8.4 Data Access Layer

Contains:

- Prisma repositories / query services
- transaction boundaries
- persistence helpers

Responsibilities:

- isolate database interactions
- keep writes consistent
- avoid duplicated query logic across controllers

8.5 Integration Layer

Contains provider adapters:

- WhatsApp mock adapter
- Meta WhatsApp Cloud API adapter
- future payment adapter
- future analytics / export adapter

Responsibilities:

- map internal domain operations to external provider APIs
- normalize provider responses
- isolate vendor-specific details

9. Product Modules

9.1 Tenant Module

Responsibilities:

- restaurant record management
- branding data
- tenant activation state
- default locale/currency/timezone settings

9.2 Branch Module

Responsibilities:

- branch listing and configuration

- manual branch selection support
- branch operating hours
- branch open/closed state
- branch-level WhatsApp mapping

9.3 Catalog Module

Responsibilities:

- categories
- products
- variants
- add-on groups
- add-ons
- availability toggles
- delivery/pickup availability

9.4 Delivery Rules Module

Responsibilities:

- delivery zones
- fees
- minimum order amount
- branch-specific coverage logic

9.5 Customer Module

Responsibilities:

- normalized phone matching
- internal customer profiles
- address capture
- repeat-order counters

9.6 Ordering Module

Responsibilities:

- cart-to-order conversion
- pricing snapshot
- fulfillment mode handling
- order numbering

- order status transitions
- audit history

9.7 WhatsApp Operations Module

Responsibilities:

- outbound order message generation
- branch routing
- interactive action mapping
- inbound webhook handling
- customer confirm/cancel messaging
- delivery of message logs

9.8 Admin Operations Module

Responsibilities:

- order visibility
- customer visibility
- menu management
- settings management
- branch management
- reporting dashboard

10. Frontend Architecture

10.1 Frontend Model

MVP frontend should be a `standalone storefront + admin dashboard` inside the same Next.js application.

This gives us:

- fastest launch path
- shared design tokens and app infrastructure
- a single deployment target
- simpler tenant branding support

10.2 Frontend Areas

```
/                       -> storefront landing
/menu                   -> menu browsing
```

```
/cart                  -> cart
/checkout              -> guest checkout
/order-confirmation    -> order placed state

/admin                 -> protected dashboard shell
/admin/orders          -> management order list
/admin/customers       -> customers
/admin/menu            -> menu management
/admin/branches        -> branch management
/admin/settings        -> restaurant and WhatsApp settings
/admin/analytics       -> basic reporting
```

10.3 Future Frontend Flexibility

The backend must remain usable by:

- NapCart-hosted storefronts
- future custom restaurant frontends
- future integrations into existing restaurant websites

That means domain logic must live in backend services, not only in UI components.

11. Backend Architecture

11.1 API Style

Use an `API-first backend inside Next.js` with:

- route handlers for public APIs
- route handlers for webhook endpoints
- protected admin APIs
- server actions only where they improve ergonomics, not as the only business interface

11.2 Why API-First Matters

This supports:

- current standalone storefront
- future existing-website integrations
- future mobile or third-party channels
- cleaner separation between UI and business logic

11.3 Core Backend Domains

```
Tenant Service
Branch Service
Catalog Service
Delivery Rule Service
Customer Service
Order Service
WhatsApp Service
Admin Reporting Service
```

12. Auth and Access Model

12.1 Customer Access

MVP customer checkout is:

- guest only
- no login
- no OTP

12.2 Admin Access

MVP admin access should use:

- `Supabase Auth` for admin authentication
- one main admin account per restaurant

Recommended model:

- Supabase Auth stores login identity
- `admin_users` table stores NapCart business association and restaurant scope

12.3 Authorization Model

MVP authorization is simple:

- admin session must resolve to one restaurant
- all admin queries must be restaurant-scoped
- branch operations must also be restaurant-scoped

Important:

- never trust client-provided restaurant IDs alone

- derive effective tenant scope from the admin session

13. Data and Persistence Architecture

13.1 Database Strategy

Use one `PostgreSQL` database for MVP.

Why:

- all main entities are relational
- reporting needs joins and filters
- transaction integrity matters during order creation

13.2 Multi-Tenant Strategy

NapCart should use `shared database, shared schema, tenant-scoped rows`.

Tenant isolation rule:

- every major operational table must contain tenant ownership directly or be reachable through tenant-owned parents

Practical model:

- `restaurant_id` on most top-level business tables
- branch-owned tables must always resolve cleanly back to the parent restaurant

13.3 Order Write Transaction

Order persistence must happen inside a transaction:

```
validate branch
validate catalog availability
validate fulfillment rules
find or create customer
create order
create order items
create order item add-ons
create initial status history
commit
then attempt WhatsApp send
```

Reason:

- the order must exist before any WhatsApp notification is attempted

13.4 Snapshotting Rule

At order time, the backend must snapshot:

- customer name
- customer phone
- branch name
- address text
- pricing totals
- ordered item names
- selected variants
- selected add-ons

This prevents later menu edits from corrupting historical orders.

14. Media and File Handling

14.1 MVP Media Scope

MVP media uploads:

- restaurant logo
- product images

14.2 Storage Choice

Use `Supabase Storage`.

Benefits:

- matches the selected data platform
- easy URL management
- suitable for product images and branding assets

14.3 Media Rules

- uploads must be validated for type and size
- stored paths should include tenant-aware structure
- public delivery URLs should be controlled and predictable

Suggested path pattern:

```
/restaurants/{restaurant_id}/branding/logo
/restaurants/{restaurant_id}/products/{product_id}/image
```

15. WhatsApp Integration Architecture

15.1 Integration Strategy

NapCart must use an adapter-based WhatsApp integration layer.

Core interface:

```
sendOrderForConfirmation(order, branchConnection)
sendCustomerConfirmation(order)
sendCustomerCancellation(order)
handleInboundWebhook(payload)
```

15.2 Provider Adapters

MVP adapters:

- `MockWhatsAppAdapter`
- `MetaWhatsAppCloudAdapter`

Why:

- development can proceed without paid API dependency
- each restaurant later can connect real credentials without redesigning order logic

15.3 Outbound Order Message Flow

```
Order persisted
  -> resolve branch
  -> load branch WhatsApp connection
  -> render structured order payload
  -> send interactive message to branch destination
  -> write outbound message log
```

15.4 Inbound Staff Action Flow

```
Staff taps Confirm or Cancel
  -> Meta webhook hits NapCart
  -> payload verified
```

```
   -> provider message mapped to order/action
   -> order status updated
   -> order_status_history inserted
   -> inbound message log stored
   -> customer notification sent
   -> outbound customer log stored
```

15.5 Message Correlation Requirement

NapCart must be able to correlate inbound WhatsApp actions to the correct order.

Required mechanisms:

- short order reference
- action payload metadata or button IDs
- persistent provider message logs

15.6 WhatsApp Reliability Rules

- do not mark provider send success without recording the provider response
- capture failures in `whatsapp_message_logs`
- allow operational review of failed sends later

16. Mock Mode Architecture

During development and early demos:

- real WhatsApp API may be absent
- order logic must still behave consistently

Mock mode behavior:

- pretend to send the order
- create outbound message log rows
- allow simulated confirm/cancel action paths
- keep the same internal service contract as the real adapter

This is mandatory for clean MVP development.

17. Order Flow Architecture

17.1 Public Ordering Flow

```
Customer enters storefront
  -> chooses branch
  -> browses menu
  -> adds items
  -> selects fulfillment type
  -> enters guest checkout details
  -> server validates branch/menu/delivery rules
  -> order transaction commits
  -> WhatsApp confirmation request is sent to branch
  -> customer sees order placed state
```

17.2 Operational Confirmation Flow

```
Branch WhatsApp receives order
  -> staff chooses Confirm or Cancel
  -> webhook updates backend
  -> customer receives matching response
  -> admin dashboard reflects final state
```

17.3 No-Response Flow

```
Order placed
  -> pending_confirmation
  -> no response from branch
  -> remain pending_confirmation
  -> dashboard shows it for management visibility
```

18. Delivery and Branch Routing Architecture

18.1 Branch Routing Model

MVP routing rule:

- customer selects branch manually
- that branch becomes the routing owner of the order

18.2 Delivery Validation Model

For delivery orders, backend validates:

- branch accepts delivery
- branch is open/accepting orders
- relevant zone exists or selected rule matches
- minimum order amount is satisfied

18.3 Pickup Validation Model

For pickup orders, backend validates:

- branch accepts pickup
- branch is open/accepting orders

18.4 Future Upgrade Path

Later, this module can support:

- area-based automatic branch selection
- coordinate-based delivery coverage
- branch load balancing

19. API Boundary Design

19.1 Public Storefront API Areas

```
GET    /api/storefront/{restaurantSlug}/branches
GET    /api/storefront/{restaurantSlug}/menu
GET    /api/storefront/{restaurantSlug}/settings
POST   /api/storefront/orders
GET    /api/storefront/orders/{publicRef}
```

19.2 Admin API Areas

```
GET    /api/admin/orders
GET    /api/admin/orders/{id}
GET    /api/admin/customers
GET    /api/admin/analytics/summary

POST   /api/admin/categories
POST   /api/admin/products
POST   /api/admin/branches
POST   /api/admin/delivery-zones
POST   /api/admin/whatsapp-connections

PATCH  /api/admin/products/{id}
```

```
PATCH  /api/admin/branches/{id}
PATCH  /api/admin/settings
```

19.3 Webhook Endpoints

```
POST /api/webhooks/whatsapp/meta
GET  /api/webhooks/whatsapp/meta
```

Purpose:

- GET handles webhook verification
- POST handles inbound events and actions

20. Background Processing Approach

20.1 MVP Position

Do not introduce a heavy queueing platform in MVP unless needed by real load.

20.2 Recommended MVP Approach

Use:

- synchronous transaction for order persistence
- immediate WhatsApp send attempt after commit
- database-backed logging for recovery and audits

Optional near-term enhancement:

- add a lightweight retry worker for failed outbound messages

21. Observability and Audit

21.1 What Must Be Logged

- order creation attempts
- order validation failures
- WhatsApp outbound sends
- WhatsApp inbound actions
- status transitions
- admin auth events

21.2 Business Audit Requirements

Critical auditable events:

- who created or triggered a status change
- when a branch confirmed or cancelled
- what was sent to WhatsApp
- what customer phone was notified

21.3 Monitoring Priorities

Monitor at minimum:

- webhook failures
- WhatsApp send failures
- order creation errors
- storage upload failures

22. Security Architecture

22.1 Core Security Rules

- all admin routes require authenticated admin sessions
- all admin reads/writes are restaurant-scoped server-side
- webhook signatures or verification tokens must be validated
- provider tokens must be encrypted at rest
- never expose service-role secrets to the client

22.2 Input Validation

Validate server-side:

- checkout payloads
- phone numbers
- branch selection
- fulfillment type
- menu item IDs and quantities
- delivery rule assumptions
- admin mutation payloads

22.3 Tenant Isolation

The most important application security risk is cross-tenant leakage.

Protection rules:

- always resolve data under the authenticated restaurant scope
- never use unscoped admin queries
- store branch ownership explicitly

22.4 Sensitive Data Handling

Sensitive fields include:

- WhatsApp access tokens
- webhook verification secrets
- admin auth identifiers
- customer phone numbers

23. Performance and Scalability

23.1 MVP Performance Priorities

- fast menu retrieval
- reliable order creation
- responsive admin dashboard filters
- low-latency webhook processing

23.2 Main Scalability Levers

NapCart can scale later through:

- read optimization and indexes
- caching storefront menu responses
- separating async messaging retries
- adding branch-aware analytics views
- partitioning heavy logs if needed later

23.3 What Not To Overbuild Now

- microservices
- event buses
- distributed queues
- advanced caching layers

24. Deployment Architecture

24.1 MVP Deployment Topology

```
Vercel
  -> Next.js application
     -> storefront routes
     -> admin routes
     -> API routes
     -> webhook routes

Supabase
  -> PostgreSQL
  -> Auth
  -> Storage
```

24.2 Environments

Recommended environments:

- local
- staging
- production

24.3 Environment Rules

- staging uses mock WhatsApp or safe test configuration
- production uses real restaurant credentials when available
- environment variables must be separated per deployment target

25. Configuration Architecture

Restaurant-specific configuration should be data-driven, not code-driven.

Config areas:

- branding
- language/currency defaults
- branch availability
- delivery/pickup enablement
- delivery zones and fees
- WhatsApp connection mapping
- minimum order amount

This is how NapCart becomes sellable to multiple restaurants without custom rewrites.

26. Existing Website Integration Path

NapCart is not only a website builder. It is an ordering platform.

Future integration options:

- link existing sites to a NapCart-hosted ordering flow
- expose menu and order APIs for custom frontends
- later provide embeddable widgets or a lightweight storefront SDK

Therefore:

- backend contracts must stay stable
- business logic must not be tightly coupled to one specific storefront theme

27. Implementation Architecture Order

Recommended build order:

```
1. Project foundation
2. Database schema
3. Admin auth and restaurant scoping
4. Catalog and branch management
5. Storefront menu and cart flow
6. Checkout and order creation engine
7. WhatsApp mock adapter
8. WhatsApp webhook and confirm/cancel flow
9. Real Meta Cloud API adapter
10. Basic analytics and operational polish
```

28. Key Architecture Decisions

28.1 Chosen Decisions

- modular monolith
- Next.js + TypeScript
- Postgres + Prisma
- Supabase for Postgres/Auth/Storage
- Vercel deployment

- API-first backend
- WhatsApp adapter abstraction
- branch-driven routing
- guest storefront
- admin-only auth
- Pakistan-first defaults with configurable expansion

28.2 Deferred Decisions

- retry worker implementation style
- exact analytics implementation detail
- CDN/media optimization strategy
- future widget SDK format

29. Risks and Mitigations

29.1 WhatsApp Interaction Complexity

Risk:

- inbound action handling can become brittle if correlation is weak

Mitigation:

- use clear order references
- log all provider message IDs
- keep webhook parsing isolated in adapter layer

29.2 Tenant Leakage Risk

Risk:

- admin queries may accidentally cross restaurant boundaries

Mitigation:

- enforce server-side restaurant scoping everywhere
- keep repository patterns explicit

29.3 Restaurant Workflow Drift

Risk:

- product may accidentally assume staff will use the dashboard actively

Mitigation:

- keep staff workflow centered on WhatsApp
- keep dashboard focused on owner/management visibility

29.4 Pakistan-Specific Data Quality

Risk:

- inconsistent phone formatting and free-text addresses

Mitigation:

- normalize phone numbers to Pakistan format
- keep address schema ready for later geo-improvement

30. Final Recommendation

NapCart should be implemented as:

- a modular monolith
- deployed on Vercel
- backed by Supabase Postgres, Auth, and Storage
- organized around domain services
- integrated with WhatsApp through a provider adapter layer

This architecture is the right MVP fit because it gives NapCart:

- fast delivery speed
- reliable operational foundations
- clean per-restaurant configurability
- a realistic path to support both new and existing restaurant websites
- future scale without rewriting the core business model